

Estruturas balanceadas aumentadas sob carga real em larga escala

Implementação, medição empírica e defesa de uma árvore de busca balanceada aumentada sobre dados reais de centenas de milhões de chaves.

§ 01

Objetivos

Ao concluir este projeto, o grupo deverá ser capaz de:

- Implementar uma estrutura de dados balanceada **aumentada**, mantendo seus invariantes sob inserção, remoção e rotação.
- Avaliar empiricamente o desempenho de inserção, remoção e busca em **larga escala** sobre dados reais.
- Confrontar o comportamento **medido** com os limites **teóricos**, explicando as divergências.
- Defender, de forma oral, as decisões de projeto e o raciocínio sobre corretude.

A ênfase recai sobre o *entendimento* e a *medição* — partes que não se resolvem copiando uma solução pronta. O artefato de código é necessário, porém é a menor fração da nota.

§ 02

Contexto

Uma árvore AVL ou um heap isolados são artefatos canônicos — existem milhares de implementações idênticas disponíveis. Este projeto evita essa armadilha de duas formas. Primeira, a estrutura é **composta e aumentada**, de modo que os invariantes precisam ser raciocinados para *aquela* combinação específica. Segunda, a maior parte da nota vem de medições que o grupo precisa **executar na própria máquina** e **interpretar com suas próprias palavras**, de uma defesa oral e de um documento de justificativa de projeto. Esses componentes resistem a soluções genéricas porque dependem do contexto particular do grupo.

O uso de ferramentas de auxílio (incluindo modelos de linguagem) é **permitido** para a parte de código. A avaliação, no entanto, concentra-se deliberadamente naquilo que essas ferramentas não conseguem produzir pelo grupo — a interpretação dos seus próprios números e a defesa do seu próprio raciocínio.

A estrutura a implementar (núcleo obrigatório)

Implemente uma **árvore binária de busca balanceada aumentada**. O balanceamento fica a critério do grupo (AVL, rubro-negra ou *treap*):

OPERAÇÃO	DESCRIÇÃO
<code>insert(k)</code>	insere a chave <code>k</code>
<code>delete(k)</code>	remove a chave <code>k</code>
<code>search(k)</code>	informa se <code>k</code> está presente
<code>rank(k)</code>	número de chaves menores que <code>k</code>
<code>select(i)</code>	a <code>i</code> -ésima menor chave (estatística de ordem)
<code>range_agg(a, b)</code>	agregado das chaves no intervalo <code>[a, b]</code>

A função de agregação de `range_agg` (soma, contagem, mínimo ou máximo) **varia por grupo** — veja a §9. O ponto difícil, e o mais avaliado, é manter os campos aumentados (tamanho da subárvore, agregado da subárvore) corretos **sob rotação**. Uma rotação que esquece de recomputar um agregado quebra todas as consultas subsequentes de forma silenciosa.

Grupos que quiserem ir além podem substituir o núcleo por uma **fila de prioridade parcialmente retroativa** — operações de inserção e remoção podem ser aplicadas "no passado", e as consultas refletem a história reescrita.

O conjunto de dados

Usaremos os conjuntos de chaves **reais** do *benchmark SOSD* (*Search On Sorted Data*). Cada conjunto contém de 200 a 800 milhões de inteiros sem sinal, extraídos de fontes reais:

CONJUNTO	ORIGEM	CARACTERÍSTICA RELEVANTE
amzn	popularidade de vendas de livros (Amazon)	distribuição moderadamente enviesada
face	identificadores de usuários (Facebook)	lacunas grandes e regulares
wiki	<i>timestamps</i> de edições de artigos	quase denso, com agrupamentos temporais
osm	dados do OpenStreetMap	lacunas muito irregulares — o mais difícil

Repositório e instruções de download: github.com/learnedsystems/SOSD

Cada arquivo SOSD é binário — 8 *bytes* iniciais com a quantidade de chaves (`uint64`, *little-endian*), seguidos das chaves (`uint32` ou `uint64`). O *script* fornecido (§6) lê esse formato diretamente.

Atenção — o SOSD é, em sua forma original, uma carga **somente de leitura**; ele não exercita inserções nem remoções. Por isso, a sequência de operações (inserção/remoção/busca) é **gerada** a partir das chaves reais pelo *script* a seguir. Como você deriva essa sequência é, em si, uma decisão de projeto avaliada.

§ 05

Como obter os arquivos

Os arquivos **não** ficam no repositório git — apenas os *scripts* de download ficam. Há duas formas:

1. Um arquivo isolado (recomendado)

Pelo espelho estável no Zenodo (CC-BY 4.0, já descomprimido):

```
wget -O osm_cellids_800M_uint64 \
  "https://zenodo.org/records/15240501/files/osm_cellids_800M_uint64?download=1"
echo "70670bf41196b9591e07d0128a281b9a osm_cellids_800M_uint64" | md5sum -c
```

Troque o nome do arquivo conforme o conjunto do grupo.

2. A suíte completa, pelo repositório

O procedimento abaixo é para uma máquina Linux baseada em apt (Ubuntu/Debian), que é o ambiente assumido pelo repositório. Requisitos — pelo menos **16 GiB de RAM** e **50 GiB de disco livre** (os dados descomprimidos somam mais de 20 GB, e o *script* ainda gera conjuntos sintéticos).

Passo 1 — Instalar as dependências. A principal para os dados é o `zstd` (os arquivos são baixados comprimidos), além de Python com numpy/scipy (usados para gerar os conjuntos sintéticos):

```
sudo apt -y update
sudo apt -y install zstd python3-pip m4 cmake clang libboost-all-dev
pip3 install --user numpy scipy
```

Se você também pretende construir os RMI's ou rodar o *benchmark* em si, instale o Rust — para apenas obter os dados isto não é necessário:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
```

Passo 2 — Clonar o repositório.

```
git clone https://github.com/learnedsystems/SOSD.git
cd SOSD
```

Passo 3 — Rodar o *script* de download a partir da raiz do repositório. É este o passo que baixa tudo. Os *scripts* devem ser executados da raiz, e não de dentro de `scripts/`:

```
./scripts/download.sh
```

O *script* baixa cada conjunto real como `.zst`, descompacta com `zstd` e roda os geradores Python dos conjuntos sintéticos.

Passo 4 — Conferir o resultado. Os arquivos são gravados no diretório `data/`:

```
ls -lh data/
```

Devem aparecer os conjuntos reais — `books_200M_uint32`, `fb_200M_uint64`, `wiki_ts_200M_uint64`, `osm_cellids_800M_uint64` — junto dos sintéticos.

Se o **download falhar** — o servidor que hospeda os dados muda de tempos em tempos e os *links* do `download.sh` ficam desatualizados periodicamente. Se o *script* falhar em alguma URL, baixe os arquivos individualmente pelo espelho do Zenodo (forma 1) e coloque-os manualmente em `data/`. O formato binário é o mesmo, então o restante do *pipeline* funciona igual.

Memória — o arquivo `osm` tem 6,4 GB. Para não carregá-lo inteiro na RAM, use a opção `--max-load` do *script* (§6), que lê apenas as primeiras N chaves direto do preâmbulo binário.

§ 06

A carga de trabalho e o oráculo (`gen_workload.py`)

O *script* `gen_workload.py` (fornecido) transforma um conjunto de chaves do SOSD em um **trace** reproduzível de operações e em um **gabarito** de respostas esperadas.

6.1 Geração do trace

```
python3 gen_workload.py generate \
  --keys osm_cellids_800M_uint64 --format sosd --key-bytes 8 \
  --max-load 200000000 \
  --out g3 --ops 500000000 \
  --universe 100000000 --mix 40:30:30 --theta 0.99 --seed 12345
```

Isso produz `g3.trace` (uma operação por linha) e `g3.expected` (gabarito das buscas):

```
I 8473625      # insere
D 19283746    # remove
S 5512837     # busca
```

```
5512837 NOT_FOUND
```

Parâmetros principais:

- `--mix I:D:S` — proporção de inserções, remoções e buscas.
- `--theta` — enviesamento Zipfiano do acesso (estilo YCSB). `0` = uniforme; `0.99` = padrão YCSB, em que poucas chaves "quentes" concentram a maior parte dos acessos.

- `--universe` — número de chaves distintas envolvidas.
- `--insert-order` — `shuffle` (padrão), `popularity` ou `sorted`. O modo `sorted` é o **caso patológico** para BSTs ingênuas — inclua-o como execução separada e explique o resultado.
- `--seed` — garante reprodutibilidade.

6.2 O oráculo de corretude

A corretude é verificada de forma **transitiva**. O resultado esperado de cada busca depende de **todas** as inserções e remoções anteriores, de modo que uma implementação incorreta de `insert` ou `delete` produz divergência em alguma busca posterior. Parte das buscas-*miss* mira deliberadamente chaves **já removidas** — assim, uma estrutura que esqueça de remover é detectada.

A estrutura do grupo deve ler o `.trace`, emitir uma linha `<chave> <FOUND|NOT_FOUND>` por operação `S`, e a saída é conferida com:

```
python3 gen_workload.py verify --expected g3.expected --candidate minha_saida.out
```

§ 07

Estudo empírico exigido

Este é o coração do projeto. Não basta "funcionar" — é preciso **medir e explicar**. No mínimo:

1. **Escala.** Meça tempo médio por operação (e percentis `p50`, `p99`) variando `n` em pelo menos quatro ordens de grandeza, até o limite que sua máquina suportar.
2. **Sensibilidade ao enviesamento.** Repita com `--theta` em `{0.0, 0.6, 0.99, 1.2}` e explique como a concentração de acessos afeta rebalanceamento e localidade de cache.
3. **Caso patológico.** Compare `--insert-order shuffle` contra `sorted`. Mostre o efeito no balanceamento.
4. **Teoria × prática.** Para cada operação, confronte o tempo medido com o limite teórico. Onde a curva real diverge da teórica, **explique por quê** (constantes, efeitos de cache, custo amortizado de rotação, alocação de memória).
5. **Linha de base.** Compare contra uma estrutura ingênua deliberada (lista ordenada, ou BST sem balanceamento) e localize o ponto de cruzamento.

Todos os gráficos devem vir das **suas** medições, identificando máquina, compilador e metodologia.

Entregáveis

#	ENTREGÁVEL	DESCRIÇÃO
1	Código-fonte	implementação completa, compilável, com instruções de execução
2	Relatório empírico	gráficos e interpretação das suas medições (§7)
3	Justificativa de projeto	por que esta estrutura, quais alternativas foram descartadas e a que custo medido; enunciado dos invariantes da árvore aumentada e argumento informal de que se mantêm sob rotação
4	Apresentação oral	~10 min; o docente fará perguntas do tipo "o que acontece se removermos esta rotação?" ou "por que seu <code>p99</code> cresce aqui?"

Parametrização por grupo

Cada grupo recebe uma combinação distinta. Nenhuma solução transfere diretamente para outro grupo, e a defesa sondará escolhas específicas da sua instância.

GRUPO	CONJUNTO	--THETA	--MIX (I:D:S)	RANGE_AGG	ORDEM DE INSERÇÃO
1	amzn	0.6	60:10:30	soma	shuffle
2	face	0.9	50:20:30	contagem	shuffle
3	wiki	0.99	40:30:30	mínimo	shuffle
4	osm	1.2	50:25:25	máximo	shuffle
5	amzn	0.99	45:25:30	máximo	sorted
6	face	1.2	40:20:40	soma	sorted
7	wiki	0.6	55:15:30	contagem	sorted
8	osm	0.9	50:20:30	mínimo	shuffle

Use sempre `--seed` igual ao número do grupo, para que os resultados sejam reproduzíveis pelo docente.

§ 10

Critérios de avaliação

PESO		COMPONENTE	O QUE SE AVALIA
20%		Implementação	corretude (verificada pelo oráculo)
30%		Estudo empírico	qualidade das medições e, sobretudo, a interpretação delas
20%		Justificativa	clareza do raciocínio sobre corretude e decisões
20%		Prompts	qualidade dos <i>prompts</i> utilizados. Enviar um <i>dump</i> de todos os chats usados, de maneira organizada. A avaliação foca no raciocínio e na iteração demonstrados, não no volume.
10%		Defesa oral	domínio demonstrado ao vivo

Observe que 80% da nota não depende apenas do código funcionar — depende de o grupo **entender e explicar** o que construiu e mediu.

§ 11

Regras

- Ferramentas de auxílio são permitidas para o código, mas todo o conteúdo entregue deve ser compreendido pelo grupo — a apresentação oral verifica isso.
- O relatório empírico deve refletir medições **reais** da máquina do grupo. Números fabricados ou inconsistentes com a metodologia descrita zeram o componente.
- Identifique claramente, no relatório, qualquer parte cuja autoria não seja do grupo.
- O grupo deve fornecer o passo a passo para a solução ser reproduzida.

§ 12

Referências

- A. Kipf et al. *SOSD: A Benchmark for Learned Indexes*. arXiv:1911.13014.
- Repositório SOSD — github.com/learnedsystems/SOSD
- B. F. Cooper et al. *Benchmarking Cloud Serving Systems with YCSB*. SoCC 2010. (distribuição Zipfiana de acessos)
- J. Gray et al. *Quickly Generating Billion-Record Synthetic Databases*. SIGMOD 1994. (geração Zipfiana)
- T. Cormen et al. *Introduction to Algorithms*, cap. sobre árvores de busca aumentadas.